

# CDM Review 2 Document: System Design

---

## Concepts of Data Mining

### 1. Data Analytics Design

---

The CDM module performs reproducible mining on a frozen corpus and returns structured JSON to a single frontend dashboard.

- **Objective:** Discover category structure, class boundaries, keyword associations, and temporal trends from unstructured news text.
- **Fixed Data Policy:** CDM uses only `frozen_corpus.csv` ; it does not use live IRT updates.

### Pipeline Diagram

```
flowchart TD
    frozen[CDM Frozen Corpus CSV] --> prep[Preprocessing and TF IDF]
    prep --> cluster[Bisecting K Means and PCA]
    prep --> classify[NaiveBayes vs LinearSVM]
    prep --> assoc[FPGrowth Rules]
    prep --> temporal[Temporal Pattern Mining]
    prep --> keywords[Keyword Prominence]
    cluster --> api[Flask CDM API]
    classify --> api
    assoc --> api
    temporal --> api
    keywords --> api
    api --> ui[CDM Frontend]
```

### Algorithm Selection and Justification

- **Bisecting K-Means + LSA:** stable clustering for sparse high-dimensional text vectors.
- **Naive Bayes vs Linear SVM:** benchmark both classifiers and keep best performer.
- **FP-Growth (not Apriori):** memory-efficient association rule mining for large text transactions.
- **PCA/SVD projection:** converts high-dimensional features to 2D visualization space.

### 2. High-Level System Architecture Design

---

The system uses a decoupled client-server architecture with strict endpoint contracts.

```
graph LR
    subgraph frontend [Frontend]
        ui[HTML JS Controller]
        charts[ChartJS Visual Layer]
    end
    subgraph backend [Flask Backend]
        router[API Router]
        cdm[CDM Engines]
    end
    subgraph dataLayer [Data Layer]
        frozenData[(frozen_corpus.csv)]
        modelArtifacts[(models and derived outputs)]
    end

    ui --> router
    router --> cdm
    cdm --> frozenData
    cdm --> modelArtifacts
    router --> charts
```

## API Specification (Current)

- GET /api/cdm/stats
- POST /api/cdm/cluster
- GET /api/cdm/elbow
- POST /api/cdm/classify
- POST /api/cdm/predict
- POST /api/cdm/pca
- POST /api/cdm/association
- POST /api/cdm/temporal
- POST /api/cdm/keywords

## 3. Database / Dataset Design

---

- **IRT storage engine:** DuckDB for dynamic and live/updating retrieval corpus.
- **CDM analysis dataset:** frozen CSV for repeatable mining and review consistency.
- **Contract:** CDM endpoints are guarded to fail if frozen corpus is missing.

## Conceptual Data Model (CDM)

```
erDiagram
    ARTICLE {
        INT doc_id
        STRING title
        STRING content
        STRING category
        STRING source
        DATE published_at
    }
    CLUSTER_RESULT {
        INT cluster_id
        FLOAT purity
        FLOAT silhouette
    }
    CLASSIFICATION_RESULT {
        STRING model_name
        FLOAT accuracy
        STRING winner
    }
    ASSOCIATION_RULE {
        STRING antecedent
        STRING consequent
        FLOAT support
        FLOAT confidence
        FLOAT lift
    }
    ARTICLE ||--o{ CLUSTER_RESULT : contributes
    ARTICLE ||--o{ CLASSIFICATION_RESULT : trains
    ARTICLE ||--o{ ASSOCIATION_RULE : derives
```

## 4. Screen / Input / Form Design

- Main interface: `index.html` with CDM sidebar modules.
- Each module has explicit run controls and structured outputs.
- New review hardening additions:
  - Status badges: `Not Run`, `Running`, `Success`, `Failed`
  - Standard output blocks: `Method`, `Input Params`, `Key Metrics`, `Interpretation`
  - Parameter hints for clustering and association thresholds
  - Review Mode toggle for compact demo flow

## 5. Visualization / Graph / Chart Design

- **Clustering:** cluster cards + purity/silhouette + PCA scatter.
- **Classification:** dual model scorecards + confusion matrices + live predict confidence.
- **Association:** rule cards with support/confidence/lift chips.
- **Temporal:** trend lines and cross-category correlation chips.
- **Keywords:** global and category-defining TF-IDF term visualizations.

## 6. Test Data and Test Case Design

---

### 6.1 Test Methodology

- Component integration tests through real API calls.
- Boundary testing for invalid params and empty result scenarios.
- Frozen-dataset contract validation.

### 6.2 Validation Snapshot

- Automated endpoint run: `python code/backend/test_api.py`
- Result: 12/12 tests passed

| Endpoint                          | Payload                                         | Expected Core Fields                       | Status |
|-----------------------------------|-------------------------------------------------|--------------------------------------------|--------|
| <code>/api/cdm/stats</code>       | none                                            | total_docs, vocabulary_size                | Pass   |
| <code>/api/cdm/cluster</code>     | <code>{"n_clusters":3}</code>                   | clusters, silhouette_score, overall_purity | Pass   |
| <code>/api/cdm/elbow</code>       | none                                            | k_values, inertia, recommended_k           | Pass   |
| <code>/api/cdm/classify</code>    | <code>{}</code>                                 | naive_bayes, svm, winner, accuracy_delta   | Pass   |
| <code>/api/cdm/predict</code>     | text input                                      | predicted_category, confidence             | Pass   |
| <code>/api/cdm/pca</code>         | <code>{"n_clusters":3,"sample_size":200}</code> | points, cluster_labels                     | Pass   |
| <code>/api/cdm/association</code> | support/confidence                              | rules, lift metrics                        | Pass   |
| <code>/api/cdm/temporal</code>    | <code>{}</code>                                 | category_trends, correlation               | Pass   |
| <code>/api/cdm/keywords</code>    | <code>{"top_n":10}</code>                       | global_top_terms, category_defining_terms  | Pass   |

## 7. Issues Faced and Resolutions

---

1. High memory pressure in large text mining
  - Resolution: feature bounds and sampling where required.
2. Review reproducibility risk from live data
  - Resolution: strict frozen-corpus-only CDM contract.
3. Frontend clarity and demo reliability
  - Resolution: status badges, standard result layout, actionable errors, review mode.

## 8. Review-2 Conclusion

---

The CDM system design is now aligned with implementation:

- correct algorithms (FP-Growth, Bisecting K-Means, SVM/NB benchmark),
- correct `/api/cdm/*` endpoint contracts,
- strict data separation from IRT live pipeline,
- review-ready frontend behavior and verifiable endpoint evidence.